



Laboratorio No.2 Semana #13 – API Secrets y autenticación

Nombre: Angel Julian Garcia Lopez **Carné:** _1097925

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

a) Abra Postman y cree una nueva colección llamada: MIA – Semana 13.

b) Cree una solicitud llamada: Login.

c) Seleccione el método POST.

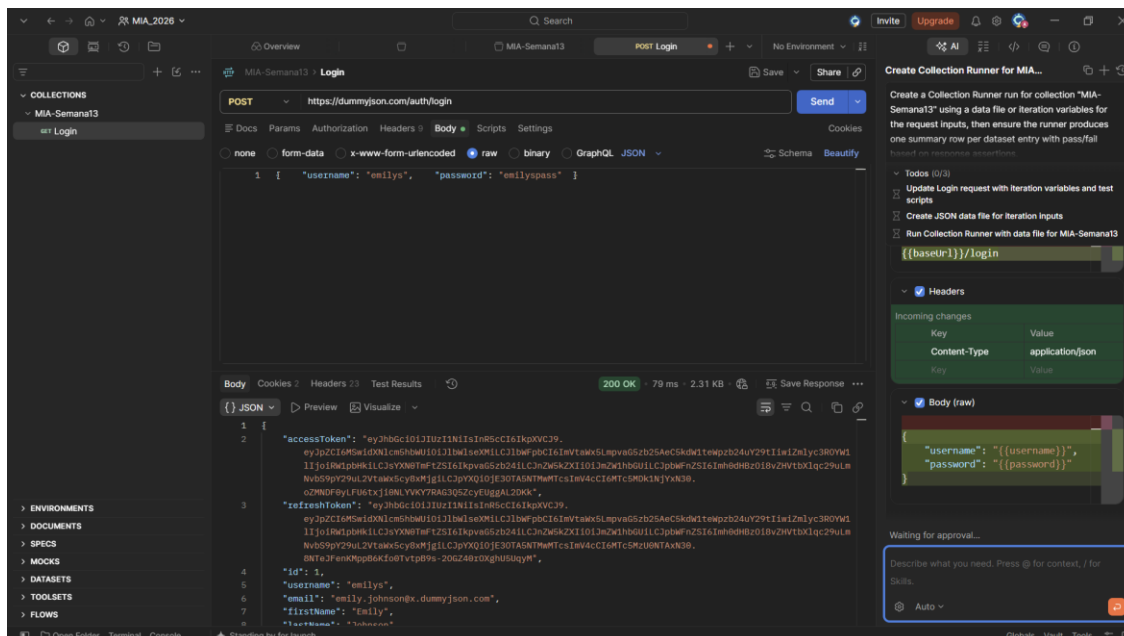
d) Utilice el endpoint: <https://dummyjson.com/auth/login>

e) En Body seleccione raw y JSON.

Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.

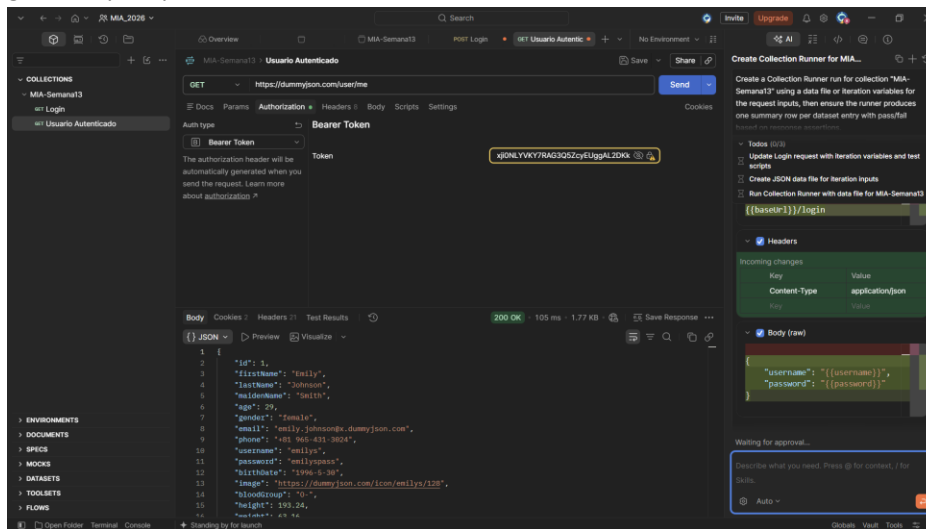
```
{ "username": "emilys", "password": "emilyspass" }
```

f) Ejecute la solicitud y localice en la respuesta el campo accessToken.



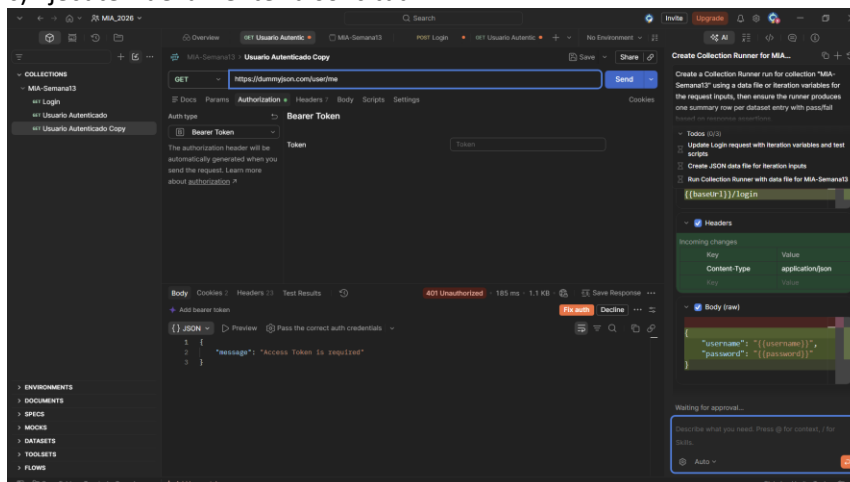
2. Utilizar token

- Cree una segunda solicitud llamada: Usuario autenticado.
- Seleccione GET.
- Utilice: <https://dummyjson.com/user/me>
- En Authorization seleccione Bearer Token.
- Copie el accessToken obtenido en la solicitud Login.
- Ejecute la solicitud.
- Verifique que la API devuelve la información del usuario autenticado.



3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.



- Compare la respuesta con la solicitud autenticada.

La solicitud autenticada devuelve correctamente la información del usuario y tiene un código **200 OK**. En cambio, al quitar el token, la solicitud devuelve un error **401 Unauthorized**, indicando que no se tienen las credenciales necesarias para acceder al recurso.

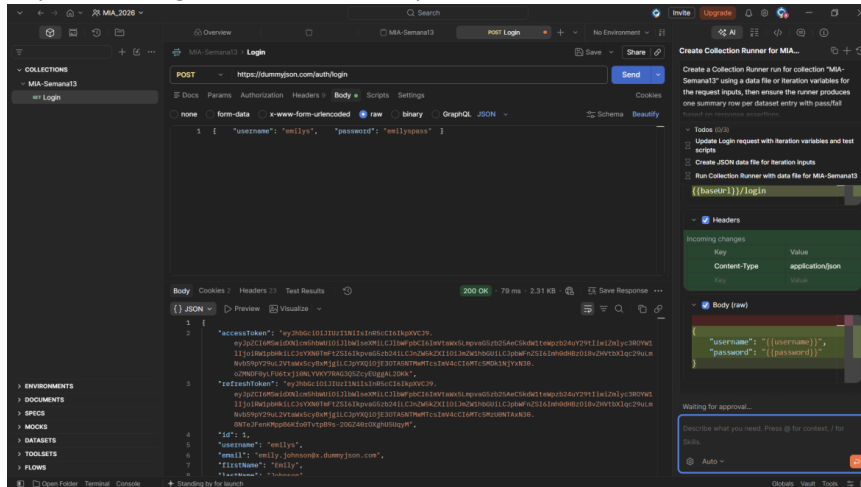


e) Registre el código HTTP y explique la diferencia.

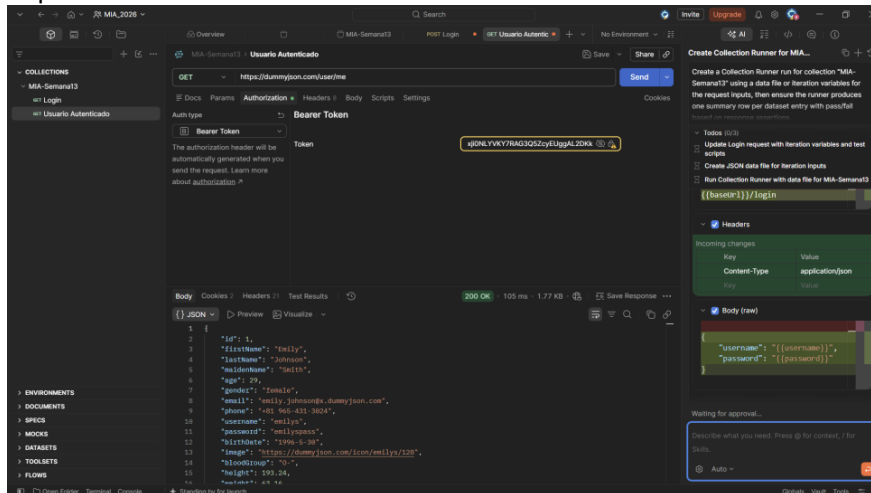
El código HTTP obtenido al quitar el token es **401 Unauthorized**. Esto significa que la solicitud no está autorizada porque no se proporcionó el token de autenticación

4. Evidencia

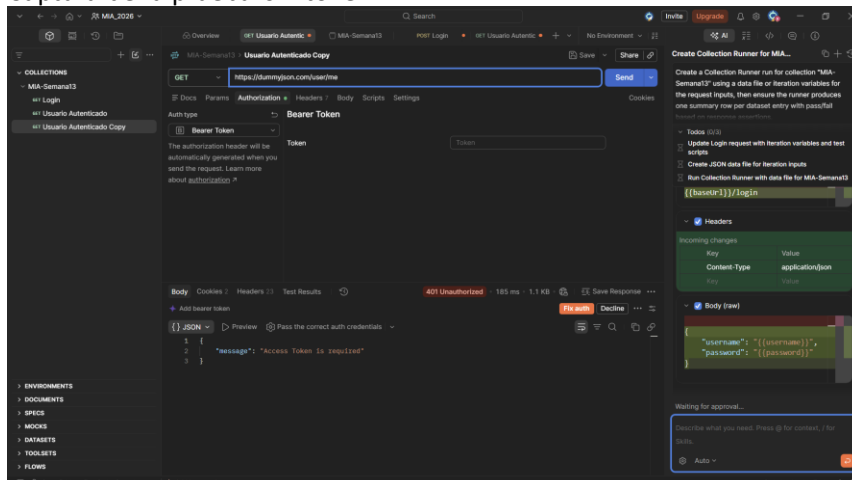
- Captura de Login mostrando la respuesta.



- Captura de la solicitud autenticada.



- Captura de la prueba sin token.





- Indique qué función cumple el Bearer Token.

El Bearer Token sirve para autenticar y autorizar al usuario ante la API. Se envía en la solicitud para demostrar que el usuario tiene permiso para acceder a la información solicitada. Si el token es válido, la API permite el acceso; si no se proporciona, devuelve un error 401 Unauthorized.

La documentación de DummyJSON muestra el flujo de login, la obtención del accessToken y su envío mediante el encabezado Authorization: Bearer.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
- En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
- Busque la variable que contiene la Connection String.
- Identifique el valor que comienza con información como DefaultEndpointsProtocol=...

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada: **AZURE_STORAGE_CONNECTION_STRING**

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

```
1 using Azure.Storage.Blobs;
2 using Azure.Storage.Blobs.Models;
3
4 references
5 class Program
6 {
7     1 reference
8     private static readonly string connectionString =
9         Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING");
10     ?? throw new Exception("No se encontró la Connection String.");
11
12     // Nombre del contenedor que creaste en Azure
13     1 reference
14     private const string containerName = "mia-archivos";
15
16     0 references
17     static async Task Main()
18     {
19         try
20         {
21             BlobServiceClient blobServiceClient =
22                 new BlobServiceClient(connectionString);
23
24             BlobContainerClient containerClient =
25                 blobServiceClient.GetBlobContainerClient(containerName);
26
27             await containerClient.CreateIfNotExistsAsync();
28
29             int opcion;
30
31             do
32             {
33                 Console.Clear();
```



7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
- Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
- Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
- Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
- Compruebe que el programa continúa funcionando correctamente.

The screenshot shows the Visual Studio Code interface. The Explorer pane on the left shows the project structure with files like .env, Program.cs, and README.md. The main editor shows the Program.cs file with the following code:

```
5 class Program
6 {
7     private static string connectionString = "";
8
9     // Nombre del contenedor que creaste en Azure
10    private const string containerName = "mia-archivos";
11
12    static async Task Main()
13    {
14        // Cargar las variables del archivo .env
15        Env.Load();
16
17        connectionString =
18            Environment.GetEnvironmentVariable("AZURE_STORAGE_CONNECTION_STRING")
19            ?? throw new Exception("No se encontró la Connection String.");
20
21        try
22        {
23            BlobServiceClient blobServiceClient =
24                new BlobServiceClient(connectionString);
25            BlobContainerClient containerClient =
26                blobServiceClient.GetBlobContainerClient(containerName);
```

The terminal at the bottom shows the output of the program execution:

```
J:\mia-2026\semana_12\Lab2\MIA_AzureBlob_AG\MIA_AzureBlob_AG.csproj".
info : Se agregó PackageReference para la versión "3.2.0" del paquete "DotNetEnv" al archivo "C:\mia_github\url-mia-2026\semana_12\Lab2\MIA_AzureBlob_AG\MIA_AzureBlob_AG.csproj".

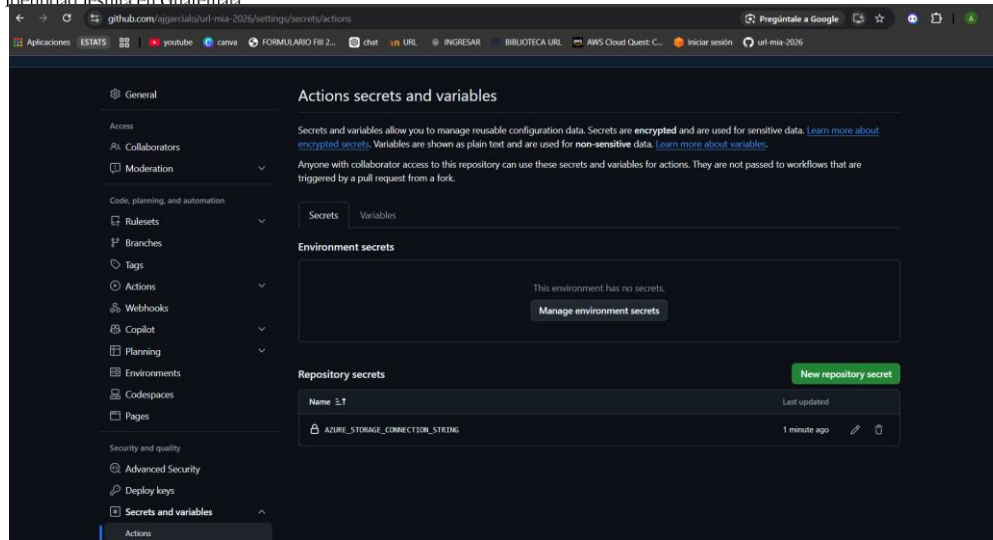
MIA - AZURE BLOB STORAGE

1. Subir archivo
2. Listar archivos
3. Descargar archivo
4. Eliminar archivo
5. Salir
```

- No publique el archivo .env en GitHub, ya que contiene la credencial real.

8. Crear el Secret en Github

- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: AZURE_STORAGE_CONNECTION_STRING
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.



Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.